TUTORIAL: Timer Interrupt

Timer and Time Delay

Name: HWANG MooYeol ID: 22000814

I. Introduction

In this lab, you will learn how to set up MCU Timers for Timer Interrupt and PWM output.

Objectives of this lab are learning how to

- Configure registers of Timers (TIMx)
- Generate Timer Interrupt

Hardware

NUCLEO -F411RE

Software

Keil uVision IDE, CMSIS, EC_HAL

Documentation

STM32 Reference Manual

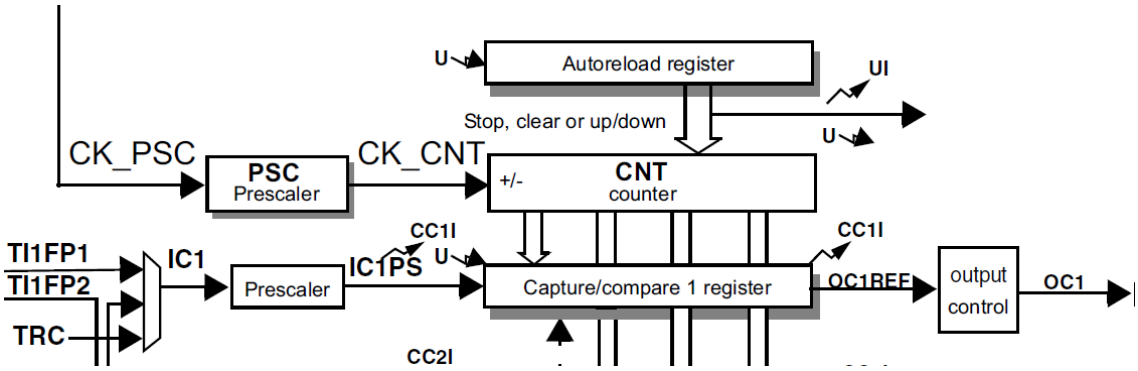
II. Basics of Timer Interrupt

A. Register List

List of TIMx registers for this tutorial.

Type	Register Name	Description
TIMx	TIMx_ CR1	TIMx control register 1
	TIMx_ PSC	TIMx prescaler register
	TIMx_ARR	TIMx auto-reload register
	TIMx_DIER	TIMx DMA Interrupt Enable register
	TIMx_BDTR	TIM1(only) break and dead-time register

Schematic



B. Register Setting

System Clock setting

1. RCC setting (PLL)

Timer setting

1. Enable Timer Peripheral Clock (**RCC→APB1ENR**)
2. Set Timer Clock Pre-scaler value (**TIMx→PSC : PSC[15:0]**)
3. Set Auto-reload value (**TIMx→ARR : ARR**)
4. Set Counting Direction (**TIMx→CR1 : DIR**)
5. Enable Timer DMA/Interrupt. **(TIMx→DIER : UIE)**
6. Enable counter (**TIMx→CR1 : CEN**)

NVIC setting

1. Enable TIMx Interrupt: NVIC_EnableIRQ(TIMx_IRQn)
2. Set interrupt Priority NVIC_SetPriority(TIMx_IRQn,2)

III. Tutorial

A. Register Configuration

Fill in the table

Port/Pin	Description	Register setting
RCC	PLL Initialization	RCC_PLL_init(); EC_SYS_CLK =EC_PLL= 84,000,000
	Enable Timer Peripheral Clock: TIM2	RCC->APB1ENR |=1≪0
TIM2	TIM2 counting direction: DIR0	TIM2->CR1 & = ~(1<<4) // upcounter
	Set Timer Clock Pre-scaler value 84MHz To 100kHz	TIM2->PSC = 0b1101000111 // 839
	Set Auto-reload value: With 100kHz, counting of 1kHz	TIM2->ARR =0b1100011 //99
	Enable Timer DMA/Interrupt	TIM2->DIER |= 1<<0
	Enable Counter	TIM2->CR1 |= 1<<0
NVIC	Set TIM2_IRQn with Priority 2, and enable	NVIC_SetPriority(TIM2_IRQn, 2); NVIC_EnableIRQ(TIM2_IRQn);

B. Programming

Procedure

- Create a new folder '**EC/Tutorial/TU_TimerInterrupt/**'
- Open the program 'Keil uVision5' and create a new project.
- Name the project as '**TU_TimerInterrupt**.
- Create a new item called '**TU_TimerInterrupt.c**'
- Use the given source code of '**TU_TimerInterrupt_student.c**' Click to download

- Fill in the empty spaces in the code.
- Run the program and check your result.
- Your tutorial report must be submitted to LMS

Exercise

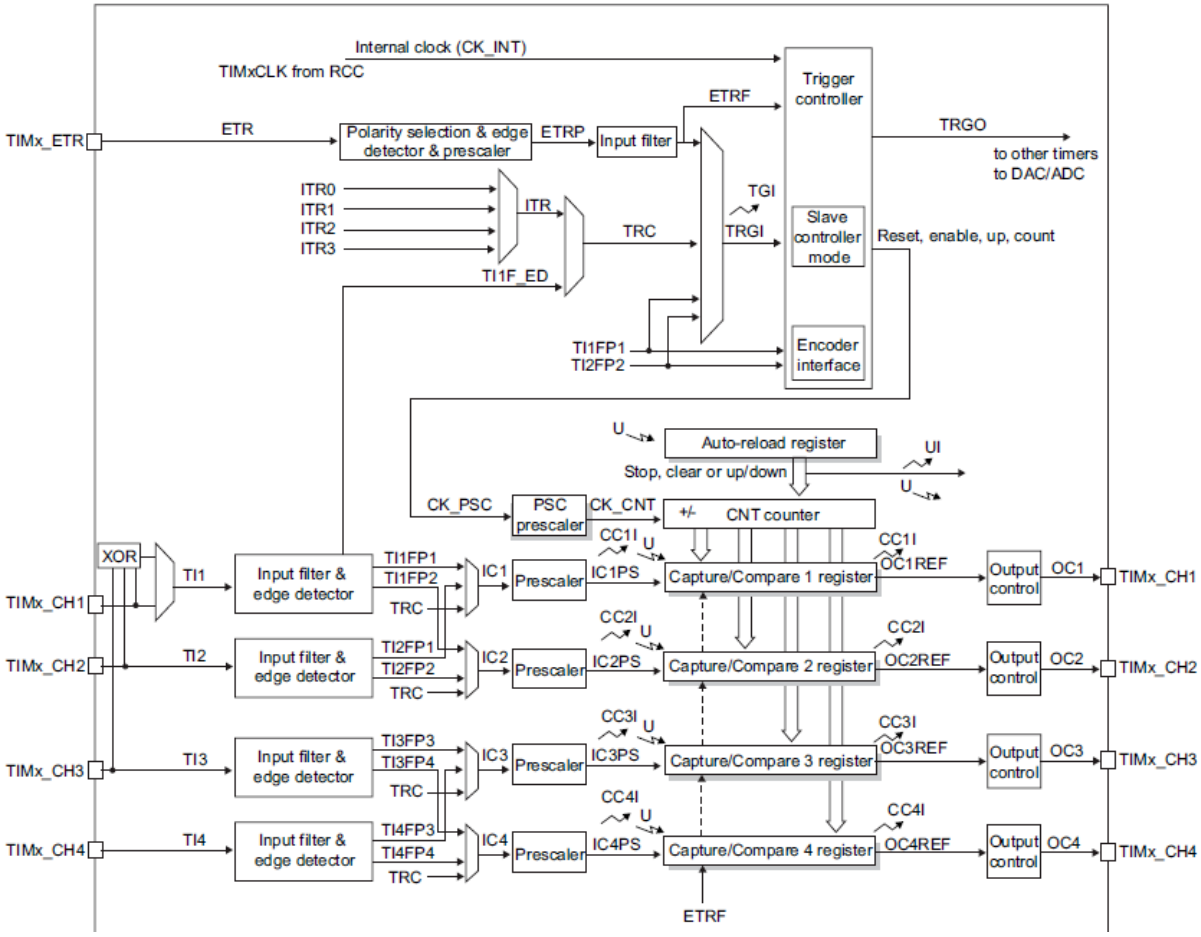
- Create a simple program that turns LED on/off at 1 second period.
- System CLK is PLL 84MHz for STM32F411RE
- Use Counter of TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK = 1 kHz
- Use timer interrupt "void TIM2_IRQHandler(void)"
- Now, change the LED blinking at 0.5 sec period

```
/**
*****************************************************************************
* @author   HWANG MooYeol
* @Mod                    2024-8-23 by YKKIM
* @brief    Embedded Controller:   Tutorial TimerInterrupt
*
*
*****************************************************************************
*/
#include "stm32f411xe.h"
#include "ecGPIO2_student.h"
#include "ecRCC2.h"


PinName_t LED_pin = PA_5;
uint32_t count = 0;


void setup(void);

int main(void) {
// Initialiization -------------------------------------------------------
setup();
TIM_TypeDef* timerx;
timerx = TIM2;
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

timerx->PSC =          0b1101000111 << 0;                    //839   Timer counter clock:
84MHz => 100kHz
timerx->ARR =          0b1100011 <<0;                        // 99     100kHz => 1kHz
timerx->DIER |= 1<<0;                                  // Enable Interrupt
timerx->CR1 |=   1<<0;                                 // Enable counter

NVIC_SetPriority(TIM2_IRQn, 2);                        // TIM2_IRQHandler Set priority as 2
NVIC_EnableIRQ(TIM2_IRQn);                             // TIM2_IRQHandler Enable


// Inifinite Loop ---------------------------------------------------------
while(1){}
}

// Initialiization
void setup(void)
{
RCC_PLL_init();                                  // System Clock = 84MHz
GPIO_init(LED_pin, OUTPUT);       // calls RCC_GPIOA_enable()
}

void TIM2_IRQHandler(void){
if((TIM2->SR & TIM_SR_UIF) ==     1     ){ // update interrupt flag
```

```
//Create the code to toggle LED by 1000ms
count++;
if (count >= 1000) {
GPIO_toggle(LED_pin);
count = 0;
}      // for 0.5sec count>=500
TIM2->SR &=     ~TIM_SR_UIF;                    // clear by writing 0
}
}
```

Appendix

[See here for MCU resources]



- TIM_TypeDef : <stm32f411xe.h>

Embedded Controller

```c
typedef struct
{
  __IO uint32_t CR1;         /*!< TIM control register 1,              Address offset: 0x00 */
  __IO uint32_t CR2;         /*!< TIM control register 2,              Address offset: 0x04 */
  __IO uint32_t SMCR;        /*!< TIM slave mode control register,     Address offset: 0x08 */
  __IO uint32_t DIER;        /*!< TIM DMA/interrupt enable register,   Address offset: 0x0C */
  __IO uint32_t SR;          /*!< TIM status register,                 Address offset: 0x10 */
  __IO uint32_t EGR;         /*!< TIM event generation register,       Address offset: 0x14 */
  __IO uint32_t CCMR1;       /*!< TIM capture/compare mode register 1, Address offset: 0x18 */
  __IO uint32_t CCMR2;       /*!< TIM capture/compare mode register 2, Address offset: 0x1C */
  __IO uint32_t CCER;        /*!< TIM capture/compare enable register, Address offset: 0x20 */
  __IO uint32_t CNT;         /*!< TIM counter register,                Address offset: 0x24 */
  __IO uint32_t PSC;         /*!< TIM prescaler,                       Address offset: 0x28 */
  __IO uint32_t ARR;         /*!< TIM auto-reload register,            Address offset: 0x2C */
  __IO uint32_t RCR;         /*!< TIM repetition counter register,     Address offset: 0x30 */
  __IO uint32_t CCR1;        /*!< TIM capture/compare register 1,      Address offset: 0x34 */
  __IO uint32_t CCR2;        /*!< TIM capture/compare register 2,      Address offset: 0x38 */
  __IO uint32_t CCR3;        /*!< TIM capture/compare register 3,      Address offset: 0x3C */
  __IO uint32_t CCR4;        /*!< TIM capture/compare register 4,      Address offset: 0x40 */
  __IO uint32_t BDTR;        /*!< TIM break and dead-time register,    Address offset: 0x44 */
  __IO uint32_t DCR;         /*!< TIM DMA control register,            Address offset: 0x48 */
  __IO uint32_t DMAR;        /*!< TIM DMA address for full transfer,   Address offset: 0x4C */
  __IO uint32_t OR;          /*!< TIM option register,                 Address offset: 0x50 */
} TIM_TypeDef;
```

```c
#define TIM2                ((TIM_TypeDef *) TIM2_BASE)
#define TIM3                ((TIM_TypeDef *) TIM3_BASE)
#define TIM4                ((TIM_TypeDef *) TIM4_BASE)
#define TIM5                ((TIM_TypeDef *) TIM5_BASE)
```

```c
/*!< APB1 peripherals */
#define TIM2_BASE           (APB1PERIPH_BASE + 0x0000UL)
#define TIM3_BASE           (APB1PERIPH_BASE + 0x0400UL)
#define TIM4_BASE           (APB1PERIPH_BASE + 0x0800UL)
#define TIM5_BASE           (APB1PERIPH_BASE + 0x0C00UL)
```